

Plantilla que debes escribir
1. Tipo: estatico / dynamic / offline / online. 2. Estructura: X porque convierte query en Y. 3. Guarda: arreglos/tablas/variantes. 4. Algoritmo: preprocess + query/update. 5. Correctitud: propiedad clave. 6. Complejidad: tiempo, memoria.
Mapa rapido de decision
Arreglo fijo, min/max Camino arbol fijo Subarbol fijo Contar rutas por arista Pares distancia $\leq C$ Texto fijo + patrones Comparar substrings Add/delete offline Forest dynamic path Forest dynamic comp/subtree Predecessor int. Sort int. sparse table LCA/HLD Euler tin/tout diff + LCA centroid SA / Aho SA+LCP+RMQ rollback DSU link-cut Euler-tour tree vEB/Y-fast/Fusion counting/radix/signature
Sparse table RMQ
Usar si arreglo no cambia y op idempotente: min/max/gcd. $st[k][i] = op(A[i..i + 2^k - 1])$ $st[k][i] = op(st[k - 1][i], st[k - 1][i + 2^{k-1}])$ Cerrado $[l, r]$: $len = r - l + 1$, $k = \lfloor \log_2 len \rfloor$ $ans = op(st[k][l], st[k][r - 2^k + 1]).$ Medio abierto $[l, r)$: $len = r - l$, segundo bloque $st[k][r - 2^k]$. Correctitud: induccion en bloques 2^k; query cubierta por 2 bloques; solape no importa por idempotencia. Costo: build/mem $O(n \log n)$, query $O(1)$. No usar: suma con solape; si hay updates usar segment tree.
RMQ pseudo
<pre>lg[1]=0; for i=2..n: lg[i]=lg[i/2]+1; st[0][i]=A[i]; for k=1..LG, for i+2^k<=n: st[k][i]=min(st[k-1][i], st[k-1][i+2^(k-1)]). query(l,r): k=lg[r-l+1]; return min(st[k][l], st[k][r-2^k+1]).</pre>
LCA binary lifting
$up[v][j] = \text{ancestro } 2^j, \quad depth[v].$ Para max edge: $mx[v][j] = \max(mx[v][j - 1], mx[up[v][j - 1]][j - 1]).$ Query u, v: subir el mas profundo, luego subir ambos de j grande a 0 si $up[u][j] \neq up[v][j]$. $dist(u, v) = dep[u] + dep[v] - 2dep[lca(u, v)].$ Costo: pre/mem $O(n \log n)$, query $O(\log n)$. Correctitud: de-composicion binaria de la distancia a subir.
HLD - Heavy Light
Arista pesada va al hijo de mayor subarbol. Arista liviana reduce tamano al menos mitad, por eso cualquier root-node cruza $O(\log n)$ livianas. Path $u \rightarrow v$ se parte en $O(\log n)$ segmentos contiguos. $\text{path query} = O(\log n) \text{ segmentos} \times O(\log n)$ con segment tree: $O(\log^2 n)$. Guarda: parent, depth, heavy, head, pos, out. Aristas: peso(centroid[v],v) se guarda en $pos[v]$; excluir $pos[lca]$ en path final. Subarbol: $[pos[u], out[u]]$.
HLD pseudo
<pre>path(a,b): res=neutral. while head[a]!=head[b]: tomar head mas profundo; res=op(res, seg.query(pos[head[a]], pos[a])); a=parent[head[a]]. Al final mismo head: query(pos[minDepth], pos[maxDepth]).</pre>
Subsecuencia en camino (lista prof.)
Enunciado: letras en vertices; query U, V, S; T=letras del camino $U \rightarrow V$; decidir si S subsecuencia de T. Diagnosticos:

arbol fijo + path ordenado + subsecuencia $\Rightarrow$ HLD + listas por caracter.
Guarda: HLD base $B[pos[v]]$; para cada char c , lista ordenada L_c de posiciones.
Query: descomponer $U \rightarrow V$ en segmentos en orden real. Consumir S greedily: buscar primera aparicion valida de $S[i]$ en segmento con lower/upper bound segun direccion.
Correctitud: HLD concatena exactamente el path; tomar primera aparicion de un caracter nunca perjudica una subsecuencia.

$T_q = O((S + \log n) \log n), \quad M = O(n).$
Subsecuencia pseudo
<pre>segments=ordered_path_HLD(U,V); i=0. for seg in segments: cur=seg.start; while i< S : p=find_first(L[S[i]],seg,cur); si no existe break; cur=next(p,dir); i++; return i== S .</pre>
Trie lexicografico (lista prof.)
Cada nodo u representa string raiz $\rightarrow u$. Ordenar nodos lexicograficamente. Solucion: DFS preorder, hijos en orden de caracter. dfs(u): output u; for c en alfabeto ordenado: if child[u][c] dfs(child). Por que: palabra prefijo < extension, entonces nodo antes que descendientes. Entre hijos, primer caracter distinto decide. Costo: alfabeto fijo $\sigma = 26$: $O(n\sigma) = O(n)$, memoria recursion $O(h)$. Si hijos desordenados: $\sum deg \log deg \leq O(n \log n)$.
Centroid: caminos peso $\leq C$ (lista prof.)

Contar pares (u, v) con $dist(u, v) \leq C$ en arbol ponderado.
No enumerar $\Theta(n^2)$. Usar centroid decomposition. En centroid c :
1) recolectar distancias $D = \{dist(c, x)\}$.
2) contar pares $D_i + D_j \leq C$ con sort+two pointers.
3) restar pares dentro de cada subarbol hijo.
4) recursar.
Correctitud: cada path se cuenta en el primer centroid que separa sus endpoints; si ambos endpoints quedan en mismo subarbol, se resta y se cuenta luego.

$T = O(n \log^2 n), \quad M = O(n).$
Centroid pseudo
<pre>solve(comp): c=centroid; all=[0]. for child part P: temp=dist(c,P); ans=count(temp); all+=temp. ans+=count(all); remove c; recurse parts. count(D): sort; two pointers; si D[l] + D[r] ≤ C: ans+ = r - l, l + +; si no r - -.</pre>
Suffix array: definiciones
Sufijo $S[i..]$. $SA[p]$=indice del p-esimo sufijo lexicografico.

$rank[i] = p \Leftrightarrow SA[p] = i.$
$LCP[p] = lcp(S[SA[p - 1]..], S[SA[p]..]).$ Ej: banana
$SA = [5, 3, 1, 0, 4, 2]$

porque *a, ana, anana, banana, na, nana*.
Rank no es letra: es posicion/clase del sufijo en orden.

Manber-Myers
Ordenar sufijos por prefijos 1, 2, 4, 8, ... Si ranks valen para longitud len, entonces sufijo i se compara por: $(rank[i], rank[i + len]).$ Ordenar pares, reasignar ranks iguales/mayores. Doblar len. Correctitud: prefijo $2len$=primera mitad len+segunda mitad len; si ranks ordenan cada mitad, el par ordena todo. Costo: $O(n \log^2 n)$ con sort general, $O(n \log n)$ con counting/radix por ranks.
Kasai + LCP RMQ
Kasai construye LCP en $O(n)$. Para i, vecino anterior: $j = SA[rank[i] - 1].$ Si antes h, al pasar a $i + 1$, h baja como mucho 1. Avanzar while chars iguales. $rank[i] = a < b = rank[j] \Rightarrow lcp(i, j) = \min LCP[a + 1..b].$ Preprocesar RMQ sobre LCP: build $O(n \log n)$, query $O(1)$.

Buscar/contar patron
Texto fijo T, patron P. Sufijos con prefijo P son intervalo contiguo en SA. L = primer sufijo $\geq P$, R = primer sufijo que no empieza con P. Existe iff $L < R$. Ocurrencias $= R - L$. Posiciones $SA[L..R - 1]$. Correctitud: orden lexicografico agrupa strings con mismo prefijo. Costo simple: $SA \tilde{O}(n \log n)$; patron $O(P \log n)$.
Subcadenas distintas
Cada substring es prefijo de algun sufijo. Sufijo $SA[i]$ aporta $n - SA[i]$ prefijos, pero $LCP[i]$ ya aparecieron con el sufijo anterior. $\#distinct = \sum_i (n - SA[i] - LCP[i]) = \frac{n(n + 1)}{2} - \sum_i LCP[i].$
Usar 64-bit.
Longest Common Substring
Para A, B: construir $T = A\#B\\$. Hacer $SA + LCP$. Revisar vecinos de origen distinto; max LCP es LCS. Correctitud: toda subcadena comun es prefijo de un sufijo de A y uno de B; en el bloque de sufijos que comparten ese prefijo hay vecinos de origen distinto. Empate lexicografico: escoger candidato de menor orden en SA. Costo: $SA + LCP$.
Comparar/ordenar subcadenas
Substrings $A = S[l_1..r_1]$, $B = S[l_2..r_2]$: $x = \min(lcp(l_1, l_2), A , B).$ Si $x < \min(A , B)$, comparar $S[l_1 + x]$ vs $S[l_2 + x]$. Si no, gana la mas corta. Si iguales, desempatar por (l, r) si statement. comparacion $O(1)$, $sort\ q$ substrings $O(q \log q)$.
Bordes de todas las subcadenas
Borde = prefijo y sufijo. Si vacio cuenta: $ans = \frac{n(n + 1)}{2} + \sum_{i < j} lcp(i, j).$
$\sum_{i < j} lcp(i, j)$=suma de minimos de todos los subarreglos de LCP. Calcular con pila monotona. Trampa: no es solo KMP de string completo si pide todas las subcadenas. Costo: $SA + LCP$ + pila $O(n)$.
Refrain / repeticion frecuente
Array/string. Maximizar $len \cdot freq$ de subcadena. $SA + LCP$. En LCP como histograma, un intervalo con minimo L y ancho w significa $freq = w + 1$. Buscar max $L(w + 1)$ con largest rectangle. Costo: $SA + Kasai + O(n)$ pila.
Periodicidad
Substring de periodo p si bloques iguales: $lcp(l, l + p) \geq len - p.$ Max $k = len/p$. Para problemas avanzados, mencionar runs: repeticiones maximales; $SA + LCP/LCS$ verifica igualdad en $O(1)$. No brute force $O(n^2)$.
Frequency of String
Para patron m y numero k, con ocurrencias ordenadas occ: $ans = \min_i (occ[i + k - 1] - occ[i] + m).$
Si $occ < k$, -1. Muchos patrones: Aho-Corasick recolecta occ; ocurrencias pueden solaparse.
Imperial Roads
Grafo ponderado conectado. Query fuerza arista $e = (u, v, w)$. MST T peso W. Agregar e crea ciclo $e + path_T(u, v)$. Quitar maxima del path: $ans = W + w - \max Edge_T(u, v).$

Correctitud: propiedad de ciclo del MST; para incluir e , el mejor arbol se obtiene rompiendo ese ciclo quitando la arista mas pesada.
Costo: Kruskal $O(m \log m)$, LCA/HLD pre $O(n \log n)$, query $O(\log n)$.

Fools and Roads
Tree, muchas paths; contar uso de cada edge. $cnt[u] ++, \quad cnt[v] ++, \quad cnt[lca(u, v)] -- = 2.$ DFS postorder: $sub[x] = cnt[x] + \sum sub[child].$

$sub[x]$ =paths que usan edge $(parent[x], x)$.
Correctitud: flujo sube desde extremos y se cancela en LCA.
Costo: LCA $O(n \log n)$, queries $O(q \log n)$, DFS $O(n)$.

DSU on tree / Tree and Queries
Para respuestas por subarbol con frecuencias/colores. Mantener hijo pesado; resolver livianos borrando, resolver pesado conservando, agregar livianos al mapa. Idea small-to-large: cada elemento se mueve a bolsa al menos doble de grande $\Rightarrow O(\log n)$ movimientos. Costo: usual $O(n \log n)$ o $O(n)$ con implementacion cuidadosa.
Paridad en caminos / palindromo
Labels en vertices. Mascara $maskRoot[v]$=xor de bits de letras desde raiz. Path u, v: $mask = maskRoot[u] \oplus maskRoot[v] \oplus bit(label[lca(u, v)])$ segun si vertices incluyen LCA. Puede formar palindromo iff: $\text{popcount}(mask) \leq 1.$

Rollback DSU offline
Operaciones add/delete/query conocidas. Convertir cada arista a intervalos de vida $[l, r]$. Insertar intervalo en segment tree de tiempo. DFS: dfs(node): snap=dsu.snapshot(); union(edges node); if leaf answer; else dfs children; rollback(snap). No path compression. Usar union by size. $T = O(m \log q \cdot T_{union}), \quad M = O(m \log q + n).$ Correctitud: camino raiz-hoja contiene exactamente aristas activas en ese tiempo.

DSU rollback con sumas
Guardar stack de cambios: parent, size, sum, components. Union a, b: root grande absorbe; $sum[root] += sum[other].$ Vertex add: guardar old $sum[root]$, sumar delta, rollback restaura. Query component sum: $sum[find(v)].$

Sin compresion para poder deshacer.
Clasificar dynamic graph
Solo insert Solo delete Add/delete offline Forest connected Forest path agg. Static path upd. General online DSU procesar reversa rollback DSU Euler-tour tree link-cut tree HLD niveles + ETT
Euler-tour tree
Representa cada arbol por secuencia Euler en BST balanceado. connected si vertices estan en mismo BST. link/cut son split/merge/splice de secuencias. Bueno para: conectividad, size/sum de componente, subtree aggregate. Costo: $O(\log n)$ por operacion si BST balanceado.

Link-cut tree
Dynamic forest con agregados en camino. Preferred paths guardadas en splay. access(v) expone camino root-v. makeroot(u); access(v) expone path $u - v$, agregado en raiz del splay. Costo: link/cut/path query $O(\log n)$ amortizado. Usar si: arbol cambia y preguntan max/min/sum en camino.

TORPEDO FINAL EDA - RESPUESTAS MODELO

Esta cara es para copiar estructura de solucion si el problema se parece.

Modelo de respuesta: problema de strings

“Construyo el suffix array del texto porque ordena todos los sufijos lexicograficamente. Las ocurrencias de un patron forman un intervalo contiguo de sufijos con ese prefijo. Para comparar sufijos arbitrarios uso el arreglo LCP y RMQ: si los ranks son $a < b$, $lcp = \min LCP[a + 1..b]$. Con esto transformo la query en busqueda binaria/comparacion de intervalos. Correctitud: el orden lexicografico agrupa prefijos iguales y el minimo de LCP entre ranks da el prefijo comun de los extremos. Complejidad: $SA\ O(n \log n)$, Kasai $O(n)$, RMQ $O(n \log n)$, query segun caso.”

Modelo: problema de static tree path

“El arbol no cambia, por eso preproceso. Si la query pide ancestros/distancia/maximo de camino, uso LCA con binary lifting guardando $up[v][j]$ y opcionalmente $mx[v][j]$. Si la query pide combinar muchos segmentos del camino con una estructura de rango, uso HLD: el camino se descompone en $O(\log n)$ segmentos contiguos. Correctitud: binary lifting descompone subidas en potencias de dos; HLD funciona porque cada arista liviana reduce el subarbol al menos a la mitad. Complejidad: LCA $O(\log n)$, HLD $O(\log^2 n)$.”

Modelo: problema dynamic offline

“Como conozco toda la secuencia de operaciones, convierto cada arista en intervalos de vida. Inserto cada intervalo en un segment tree sobre el tiempo. Recorro el segment tree con DFS y un DSU con rollback. Al entrar a un nodo aplico sus uniones; en una hoja respondo las queries de ese instante; al salir hago rollback al snapshot. Correctitud: las aristas activas en un tiempo t son exactamente las que estan en los nodos del camino raiz-hoja t . Complejidad: cada intervalo aparece en $O(\log q)$ nodos, por tanto $O(m \log q)$ uniones y memoria $O(m \log q + n)$.”

Modelo: problema integer predecessor

“Defino n llaves, palabra de w bits y universo $u = 2^w$. En comparaciones, predecessor cuesta $O(\log n)$, pero en word RAM puedo usar bits. vEB divide el universo en high-/low de tamano $\sqrt{u}$, con recurrencia $T(u) = T(\sqrt{u}) + O(1) = O(\log \log u)$, aunque usa $\Theta(u)$ espacio. Si necesito espacio lineal, uso Y-fast: representantes mas buckets de tamano $\tilde{O}(w)$, operaciones $O(\log w)$ esperadas/amortizadas y espacio $O(n)$.”

Modelo: problema de conteo de pares en arbol

“No puedo enumerar los $\Theta(n^2)$ pares. Uso centroid decomposition. En cada centroid cuento todos los pares cuyo camino pasa por el centroid usando distancias ordenadas y dos punteros. Resto los pares que estan dentro del mismo subarbol hijo porque esos no pasan por el centroid. Luego recursivamente resuelvo cada componente. Correctitud: cada camino tiene un primer centroid en la recursion que separa sus endpoints; se cuenta exactamente ahi. Complejidad $O(n \log^2 n)$, memoria $O(n)$.”

Modelo: problema de MST forzado

“Primero calculo un MST T de peso W . Para forzar una arista $e = (u, v, w)$, observo que $T + e$ crea un unico ciclo: e mas el camino entre u y v en T . Para obtener un arbol que incluya e , debo quitar una arista de ese ciclo; el costo minimo se logra quitando la de mayor peso del camino. Por tanto $ans = W + w - \max Edge_T(u, v)$. Preproceso maxEdge con LCA/HLD. Correctitud por propiedad de ciclo del MST.”

Modelo: problema de trie lexicografico

“Cada nodo representa la cadena desde la raiz. En orden lexicografico, una cadena va antes que cualquiera de sus extensiones propias, por eso visito primero el nodo y luego sus descendientes. Entre hijos de un mismo nodo, el primer caracter distinto decide, por eso los recorro en orden creciente. El algoritmo es DFS preorder con

hijos ordenados. Si el alfabeto es constante, el tiempo es $O(n)$; si debo ordenar hijos, $O(n \log n)$.”

Modelo: subsecuencia en camino

“El arbol es estatico y la query es sobre un camino ordenado, por eso uso HLD. Linealizo el arbol y guardo, para cada caracter, las posiciones donde aparece. Descompongo el camino $U \rightarrow V$ en segmentos HLD en el orden real. Recorro la cadena S y busco greedily la primera aparicion valida del caracter actual dentro de los segmentos usando binary search. Greedy es correcto porque tomar una aparicion mas temprana deja un sufijo del camino igual o mas largo para los caracteres restantes. Complejidad $O((|S| + \log n) \log n)$.”

Modelo: metricas

“Para probar que d es metrica verifico no negatividad, identidad, simetria y triangular. Para refutar, basta un contraejemplo, usualmente a triangular. Ejemplo: $(x - y)^2$ falla con 0, 1, 2, porque $4 > 1 + 1$. La suma de metricas es metrica porque cada propiedad se preserva; la triangular sale sumando las dos desigualdades triangulares.

Funciones como L_1, L_2, L_∞ , pesos positivos de L_1 , $\sqrt{|x - y|}$, $\min(1, |x - y|)$ son metricas por desigualdades conocidas/subaditividad.”

Tabla ultracorta de complejidades

Sparse table	build/mem $n \log n$, q 1
LCA	pre/mem $n \log n$, q $\log n$
HLD	pre n , q $\log^2 n$
SA+Kasai	$n \log n + n$
LCP RMQ	build $n \log n$, q 1
Centroid	$n \log^2 n$, mem n
Rollback DSU	$m \log q$ intervals
ETT/LCT	$\log n$ amort/op
vEB	$\log \log u$, mem u
Y-fast	$\log w$, mem n
Radix	$nw / \log n$

Si no sabes que hacer

1. Si es string: define sufijo, SA, rank, LCP. Intenta convertir a intervalo, suma LCP o comparacion LCP. 2. Si es arbol fijo: LCA/HLD/Euler timestamps. 3. Si es muchas rutas: diferencia en arbol o centroid. 4. Si hay add/delete y se puede offline: rollback DSU. 5. Si dice integer/predecessor/successor: declara n, w, u , compara BST vs vEB/Y-fast. 6. Siempre termina con correctitud + complejidad.

Mini ejemplo para explicar rank

En banana, sufijos ordenados:

5 : a , 3 : ana , 1 : $anana$, 0 : $banana$, 4 : na , 2 : $nana$.

SA = [5, 3, 1, 0, 4, 2], rank[1] = 2, rank[0] = 3.

Si comparo sufijos 1 y 3, ranks 2 y 1. El LCP se obtiene con RMQ en LCP entre esas posiciones.

Mini ejemplo Fools

Path $u - v$. Sumar +1 en u, v , -2 en lca . Al acumular postorder, si una arista separa un endpoint del resto del camino, queda flujo positivo; fuera del camino, las contribuciones se cancelan.

Mini ejemplo centroid

Centroid c . Distancias $D = [0, 1, 2, 5, 7]$, $C = 7$. Two pointers cuenta pares suma ≤ 7 . Si ambos nodos estan en el mismo child-subtree, ese par no pasa por c ; se resta y se resolvera dentro de ese child.

Mini ejemplo radix

Enteros de $w = 32$, n grande. Elegir digito $b = \log_2 n$. Cada pasada counting cuesta $O(n + 2^b) = O(n)$. Numero de pasadas $32/b$. Total $O(nw / \log n)$.

vEB/Y-fast/Fusion compacto

vEB: $x = (high(x), low(x))$, clusters de tamano $\sqrt{u}$, guarda $min, max, summary, clusters$. Successor: si $x < min$ devuelve min ; si hay sucesor en cluster, combinar high+low; si no, buscar siguiente cluster en summary. $T(u) = T(\sqrt{u}) + O(1) = O(\log \log u)$, memoria $\Theta(u)$. **Y-fast**: representantes + buckets $O(w)$: $O(\log w)$, memoria $\tilde{O}(n)$. **Fusion**: bits criticos/sketches, $O(\log_w n)$.

Integer sorting compacto

Counting: $O(n + U_d)$ tiempo y $O(U_d)$ espacio. **Radix**: digitos de b bits, estable:

$$O((n + 2^b) \lceil w/b \rceil).$$

Con $b = \Theta(\log n)$: $O(nw / \log n)$; si $w = O(\log n)$, lineal. **Signature**: para w grande, reducir a firmas/hashees de $\tilde{O}(\log n)$ y luego packed/radix.

Mas formulas que se copian

$$\text{Sparse: } k = \lfloor \log_2(r - l + 1) \rfloor.$$

$$\text{Distinct substr: } n(n + 1)/2 - \sum LCP.$$

$$\text{Forced MST: } W + w - \max Edge_T(u, v).$$

$$\text{Path count diff: } +1, +1, -2 \text{ en LCA.}$$

$$\text{Radix: } O(nw / \log n), \quad \text{vEB: } O(\log \log u).$$

Mini pruebas copiables

Sparse: induccion en k , query cubierta por 2 bloques, solape ok por idempotencia. **SA search**: strings con prefijo P forman intervalo contiguo. **LCP RMQ**: minimo entre ranks limita y garantiza prefijo comun. **MST forced**: propiedad de ciclo. **Fools**: flujo se conserva y cancela en LCA. **Centroid**: primer centroid que separa endpoints cuenta una vez. **Rollback**: camino raiz-hoja tiene exactamente aristas activas.

Errores que matan puntos

Confundir $[l, r]$ con (l, r) . No mencionar memoria. Usar path compression con rollback. En HLD de aristas, olvidar excluir LCA. Decir solo “suffix array” sin explicar intervalo/LCP. Decir vEB espacio $O(n)$ cuando es $\Theta(u)$. Usar int en valores $\Theta(n^2)$. No justificar por que evitas brute force $O(n^2)$.

Frases para cerrar correctitud

“La query se transforma exactamente porque...” “El invariante mantenido es...” “Cada objeto se cuenta una sola vez...” “La estructura contiene exactamente las aristas activas/el camino/el intervalo...” “La propiedad de ciclo/interv contiguo/mitad pesada garantiza la respuesta.”